

Hackathon Presentation: Questions and Answers

Project: Mini ERP / Inventory & Sales Management (Hackathon Entry)

General

1. Q1. What problem does the project solve?

It provides a lightweight ERP focusing on inventory, sales, and delivery workflows to help small businesses manage stock, orders, and customers with minimal setup.

2. Q2. Who is the target user?

Small-to-medium retailers, distributors, and anyone needing simple inventory and sales tracking without heavy ERP overhead.

3. Q3. What is the elevator pitch?

A fast-to-deploy mini-ERP that manages products, inventory movements, sales orders, and deliveries with role-based access and clear dashboards for operational teams.

Setup & Getting Started

4. Q1. How do you set up the project locally?

Clone the repo, install backend dependencies (npm), configure environment variables (DB URL, JWT secret), seed data, then run the backend and frontend dev servers as documented in README.

5. Q2. What are required system dependencies?

Node.js (≥ 16), npm or pnpm, a MongoDB or configured DB, and Python if running helper scripts used in docs generation.

Backend Architecture

6. Q1. What is the backend stack?

Node.js with Express, modular controllers/services structure, and a database layer managed in ``src/config/db.js``.

7. Q2. How are routes organized?

Routes are grouped by resource under ``src/routes`` (auth, product, inventory, sales, delivery, user). Each route calls controllers which use services for business logic.

8. Q3. How is business logic separated?

Controllers handle request/response, services encapsulate domain logic, and models define data shapes.

Frontend & UX

9. Q1. What frontend tech is used?

A Vite + React setup with Tailwind CSS for styling. Entry points are in ``frontend/src``.

10. Q2. How are API calls handled?

There is a shared ``axiosInstance`` and resource-specific API modules under ``frontend/src/api`` for consistent requests and error handling.

APIs & Data Flow

11. Q1. What are key API endpoints?

Auth (``/auth``), Products (``/products``), Inventory Movements (``/inventory-movements``), Sales Orders (``/sales``), Deliveries (``/deliveries``), Users (``/users``).

12. Q2. How does the product availability check work?

``productAvailability.service.js`` computes availability by subtracting committed quantities from on-hand inventory based on movements and pending orders.

Database & Persistence

13. Q1. Which database is used?

The project can run on MongoDB or a relational DB if adapters are provided; models are in ``src/models`` and are designed for simple document-style storage.

14. Q2. How is data seeded?

``seeders/seed.js`` contains sample data generation for products, users, customers, and sample sales to bootstrap demos.

Security & Access Control

15. Q1. How is authentication implemented?

JWT-based authentication with tokens issued at login; middleware in ``auth.middleware.js`` protects routes.

16. Q2. What about authorization?

Role-based access control is implemented via ``rbac.middleware.js`` and ``constants/permissions.js`` maps roles to permissions.

Scalability & Performance

17. Q1. Can this scale for more users?

Yes—by moving the database to managed clusters, adding load balancing, and separating services into microservices if needed.

18. Q2. What are current performance considerations?

The service layer is synchronous and single-process; for high throughput, add caching, optimize DB queries, and adopt pagination for heavy endpoints.

Testing & QA

19. Q1. What tests exist?

The repository includes unit-like structures and validators; adding automated tests (Jest/Mocha) is recommended for CI.

20. Q2. How to test APIs manually?

Use provided Postman collection in ``backend/docs/mini_erp_phase_2.postman_collection.json`` to run integration flows.

Deployment & DevOps

21. Q1. How to deploy to production?

Containerize services with Docker, deploy to a cloud provider (AWS/GCP/Azure), configure environment secrets, and use a managed DB.

22. Q2. What CI/CD flow is recommended?

Run tests, lint, build frontend, then deploy artifacts via GitHub Actions or another CI tool, with database migrations handled during release.

Demo & Presentation

23. Q1. What should be highlighted in the demo?

User flow: login as manager, create a product, adjust inventory, create a sales order, and complete delivery. Show dashboards reflecting changes.

24. Q2. What demo data to prepare?

Seed a few customers, products with different stock levels, and an example sales order to showcase edge cases like low stock.

Product & Business

25. Q1. What is the business value?

Reduces manual inventory errors, speeds up order fulfillment, and provides visibility into stock and sales—improving operational efficiency.

26. Q2. What is the pricing or go-to-market idea?

Offer a freemium tier for small stores and paid plans for advanced integrations, analytics, and multi-location support.

Team & Process

27. Q1. Who built the project and roles?

A small agile team: frontend dev, backend dev, product/UX, and QA/tester. During the hackathon we split tasks for rapid iteration.

28. Q2. How long did it take?

MVP was built in the hackathon timeframe (48–72 hours) with additional refinements afterwards.

Challenges & Tradeoffs

29. Q1. What's the hardest technical challenge?

Designing product availability correctly across concurrent orders and inventory movements to avoid overselling.

30. Q2. What tradeoffs were made?

Prioritized rapid delivery and simplicity over exhaustive features (e.g., complex multi-warehouse logic).

Future Work

31. Q1. What features are planned next?

Multi-location support, barcode/RFID integrations, supplier purchase orders, and automated reorder rules.

32. Q2. How to make it enterprise-ready?

Add audit trails, stronger RBAC, multi-tenancy, analytics dashboards, and SLA-backed hosting.

Technical Deep-Dive (Possible Interview Questions)

33. Q1. Explain how `inventoryMovement` works end-to-end.

Inventory movements track inbound and outbound operations; each movement updates inventory records and reconciles with pending sales to update availability.

34. Q2. How does the system prevent race conditions when two users sell the last unit simultaneously?

Implement DB-level transactions or optimistic concurrency control with versioning; currently the demo uses application-level checks and should be enhanced for production.

35. Q3. How are validations handled?

Validators in `src/validators` centralize input checks; controllers call them before service logic to ensure data integrity.

36. Q4. How would you add background jobs?

Introduce a message queue (Redis/RabbitMQ) and worker processes (BullMQ) for long-running tasks like sending emails or reconciling stock.

Non-Technical & Stakeholder Questions

37. Q1. How does this help a store owner day-to-day?

Simplifies stock checks, minimizes stockouts, speeds up order processing, and provides simple reporting for decision-making.

38. Q2. What training is required for staff?

Minimal: basic navigation and order processing training (under 1 hour) and a short onboarding guide.

39. Q3. How do you onboard a new merchant?

Import product list, set initial stock via CSV or seed, configure users and roles, and run a short training/demo.

Risks & Mitigation

40. Q1. What are the main risks?

Data loss, overselling, security vulnerabilities, and insufficient monitoring in production.

41. Q2. How to mitigate them?

Implement backups, DB transactions, input validation, security best practices, and monitoring/alerts.

Miscellaneous

42. Q1. Why not use an existing ERP?

Existing ERPs can be heavy and costly; this solution targets quick setup and core inventory/sales needs without unnecessary complexity.

43. Q2. Is the project open source?

Yes—license details are in the repo; contributors can extend modules and submit pull requests.